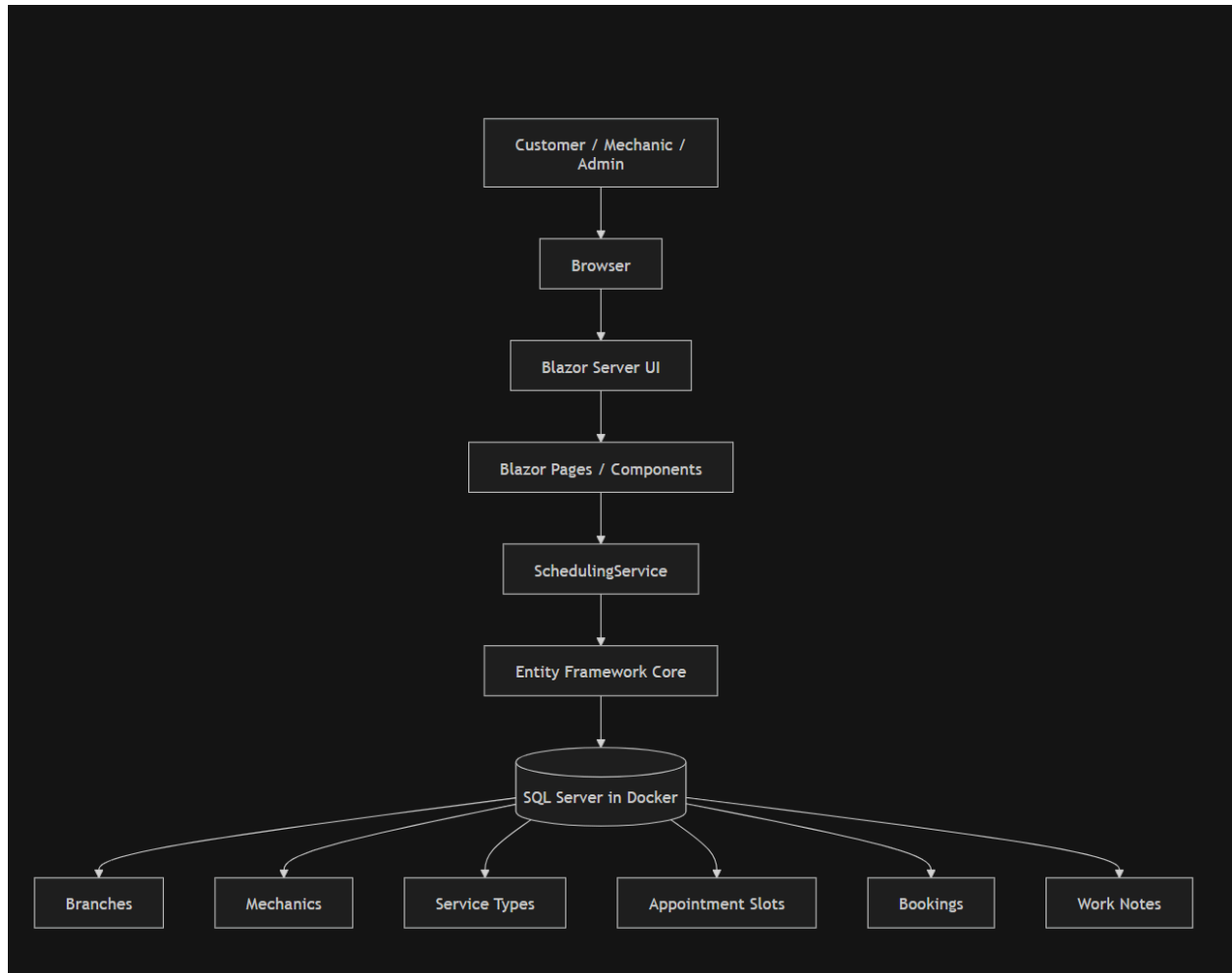


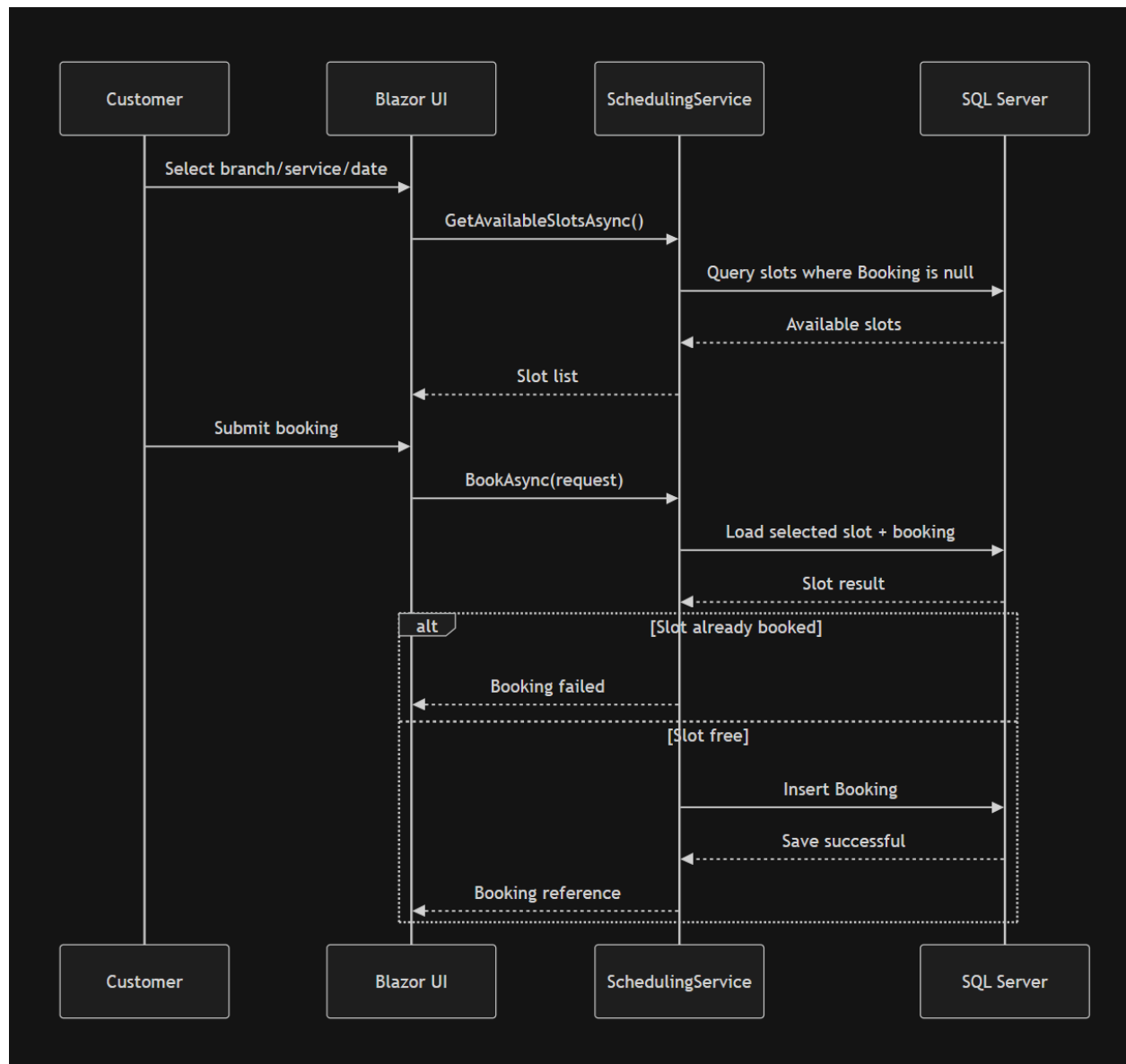
Design Decision

The design is intentionally simple: Blazor Server handles the UI, SchedulingService owns the business logic, EF Core maps the domain model, and SQL Server enforces data integrity with unique constraints to prevent double booking.

Tech Design



Booking Flow Diagram



DB Diagram

Branch (Id, Name, Address)

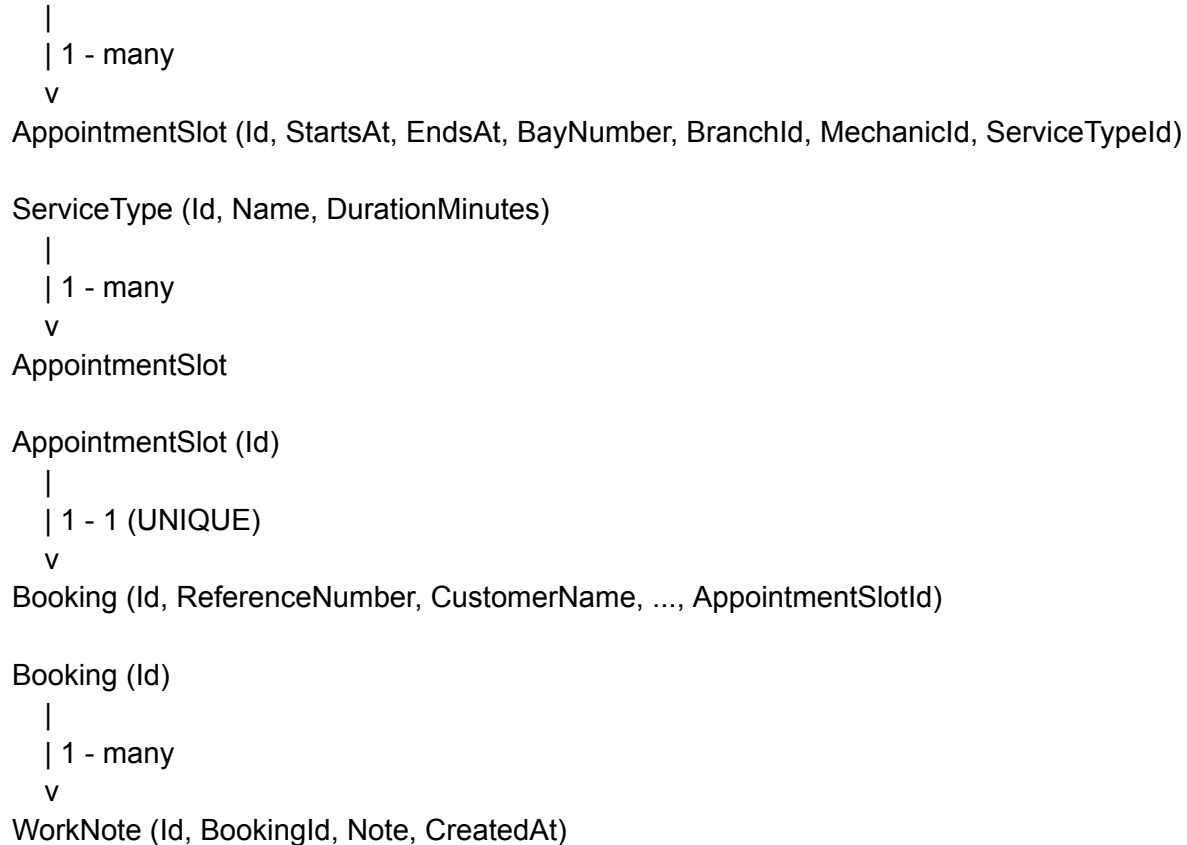
|

| 1 - many

v

Mechanic (Id, Name, BranchId)

Branch (Id)



1. High-level architecture walkthrough

This is a small Blazor Server application built on .NET 8 with Entity Framework Core and SQL Server. I kept it intentionally simple for the interview task: one web project, EF entities for the core scheduling domain, a `SchedulingService` for application logic, and Blazor pages for customer booking, mechanic views, appointment details, and the shared schedule.

The main flow is: the user filters available slots by branch and service type, chooses a slot, and creates a booking. Mechanics can then view today and tomorrow's appointments, open appointment details, update status, and add timestamped work notes.

The database model is based around `Branch`, `Mechanic`, `ServiceType`, `AppointmentSlot`, `Booking`, and `WorkNote`. I used database-level unique constraints to prevent double booking of a slot and to keep booking references unique.

Evidence: README says it uses Blazor Server, .NET 8, EF Core and SQL Server, and lists slot search, booking, double-booking prevention, mechanic views, work notes, status updates, and shared schedule.

2. Key technical decisions

I chose Blazor Server because it keeps the task compact in one .NET project while still showing the requested .NET and SQL stack.

I put most business operations into SchedulingService instead of spreading logic across UI pages. That keeps the pages thinner and makes booking, availability, status updates, and work notes easier to reason about.

For double-booking prevention, I used a unique index on Booking.AppointmentSlotId. The UI filters only free slots, but the database constraint is the real protection if two users try to book the same slot at the same time.

I used EnsureCreated and seed data because this was an interview exercise and I wanted the app to run quickly in a clean environment. For production, I would replace that with proper EF migrations.

The unique indexes are in `SmrDbContext`: booking reference is unique, `AppointmentSlotId` is unique, and branch/start/bay is unique for slots.

3. AI usage answer

I used Codex as an accelerator, not as the owner of the design. I used it to read the brief, propose a simple architecture, scaffold the Blazor and EF code, and help write the README. I then reviewed the structure, narrowed the scope, and made manual decisions around what belonged in the MVP.

Example prompt 1:

“Build a compact .NET 8 Blazor Server appointment scheduler for an SMR workshop using EF Core and SQL Server. Include branches, mechanics, service types, appointment slots, bookings, mechanic notes, and status updates.”

Example prompt 2:

“Review this booking flow and suggest how to prevent double booking when two users select the same appointment slot.”

Example prompt 3:

“Create a short README explaining how to run the app, what is done, what is out of scope, and what I would improve with more time.”

The README already states Codex was used to read the brief, plan implementation, generate initial Blazor/EF code, and write the README.

4. Likely live-change questions

They may ask you to add one of these:

- Add cancellation status
- Add reschedule appointment
- Add filter by mechanic
- Add validation for phone/name/vehicle registration
- Add search by booking reference
- Add appointment history for a mechanic
- Add customer cancellation notes
- Add audit trail for status changes
- Add seeded data for more branches or services

Best one to be ready for: **cancellation**.

Say:

I would add Cancelled to AppointmentStatus, expose a Cancel button in appointment details, then call the existing UpdateStatusAsync method. If we wanted the slot to become bookable again, I would need to decide whether cancelled bookings still occupy the slot or whether availability should ignore cancelled bookings.

Current statuses are **Scheduled**, **InProgress**, **Completed**, and **NoShow**.

5. Short script to say in interview

I built this as a compact SMR appointment scheduler using Blazor Server, .NET 8, EF Core and SQL Server. The core domain is simple: branches have mechanics, mechanics have appointment slots, customers create bookings, and mechanics manage appointment status and work notes.

The main business logic is centralized in SchedulingService. Availability queries return unbooked slots for the next seven days, with optional branch and service filters. Booking creates a booking against a selected slot, and double-booking is protected both in code and at the database level using a unique index on AppointmentSlotId.

For the interview scope I deliberately kept authentication, payments, notifications, cancellation and rescheduling out. In production, I would add EF migrations, stronger validation, integration tests, audit trails, authentication, and cancellation/rescheduling flows.

Use this. It is clean and credible.

